

CS 3513: Programming Languages

Programming Project Report

Group of 2 Students

RPAL Interpreter

Lexical Analyzer, Parser, Standardizer & CSE Machine

230427V T. P. Nawarathna

230653J K. M. H. B. M. Udurawana

May 22, 2026

Contents

1	Introduction	2
2	Project Structure	2
3	Architecture Overview	3
4	Module Descriptions and Function Prototypes	3
4.1	4.1 Entry Point: <code>rpal20.java</code>	3
4.2	4.2 Lexical Analyzer: <code>src/lexer/Lexer.java</code>	4
4.3	4.3 Parser: <code>src/parser/Parser.java</code>	5
4.4	4.4 AST Node: <code>src/parser/ASTNode.java</code>	6
4.5	4.5 Standardizer: <code>src/standardizer/Standardizer.java</code>	7
4.6	4.6 CSE Machine: <code>src/cse/CSEMachine.java</code>	8
4.7	4.7 Tree Printer: <code>src/util/TreePrinter.java</code>	12
5	Built-in Functions	13
6	Build and Run	13
6.1	6.1 Building	13
6.2	6.2 Running	13
6.3	6.3 Make Targets	14
7	Sample Program and Output	14
8	Testing	15

Introduction

This report describes the design and implementation of an RPAL (Right-reference Pedagogical Algorithmic Language) interpreter written in Java. The interpreter accepts an RPAL source file and executes it by chaining four phases:

1. **Lexical analysis:** tokenises the raw source.
2. **Parsing:** builds an Abstract Syntax Tree (AST).
3. **Standardization:** transforms the AST into a Standardized Tree (ST) using eight rewrite rules.
4. **CSE Machine:** evaluates the ST on a Control-Stack-Environment (CSE) machine; output matches `rpal.exe` exactly.

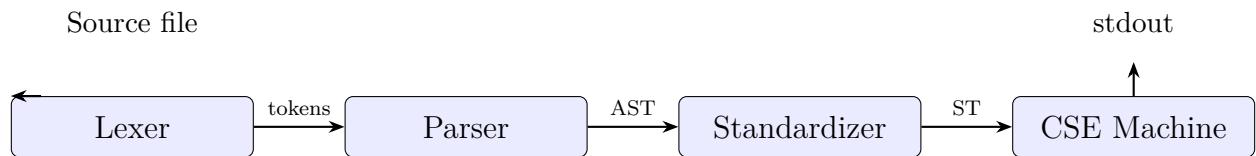
The entry point is `rpal20.java`. No external parser generators (`lex`, `yacc`, `ANTLR`, etc.) were used.

Project Structure

RPAL-Interpreter/

```
rpal20.java  Entry point, CLI arg parsing
Makefile    Build rules
src/
  lexer/
    TokenType.java  Token-kind enum
    Token.java      Immutable token value object
    Lexer.java      Hand-written lexer
  parser/
    ASTNode.java    Tree node (label + children list)
    Parser.java      Recursive-descent parser
  standardizer/
    Standardizer.java  Post-order AST to ST transformer
  cse/
    CSEMachine.java   Flatten ST and run CSE evaluation
  util/
    TreePrinter.java   Dotted pre-order tree printer
out/    Compiled .class files (after build)
rpal_test_programs/  Sample RPAL programs
```

Architecture Overview



Module Descriptions and Function Prototypes

4.1 Entry Point: rpal20.java

The `main` method parses CLI switches, reads the source file, and chains all four phases.

```
1 public class rpal20 {
2     /**
3      * CLI: java rpal20 [-l] [-ast] [-sast/-st] [-cse] <file>
4      * -l      echo source before running
5      * -ast    print AST (dotted pre-order) then exit
6      * -sast   print Standardized Tree then exit
7      * -cse    print control structures + step trace then
8      *          run
9      */
10    public static void main(String[] args) throws Exception
11 }
```

Listing 1: rpal20.java – main method

Execution flow:

1. Parse `args`: set boolean flags and extract file path.
2. Read file bytes -> `String src`.
3. `Lexer lex = new Lexer(src);`
4. `ASTNode ast = new Parser(lex).parse();`
5. If `-ast`: `TreePrinter.print(ast, 0)` then return.
6. `ASTNode st = Standardizer.standardize(ast);`
7. If `-sast`: `TreePrinter.print(st, 0)` then return.
8. `CSEMachine m = new CSEMachine(st); m.run();`

4.2 Lexical Analyzer: `src/lexer/Lexer.java`

A hand-written, single-pass lexer with one-token look-ahead. Whitespace and `//` comments are skipped implicitly.

```

1 public class Lexer {
2
3     // Constructor: accepts the full source string
4     public Lexer(String src)
5
6     // Return next token without consuming it (one-token look
7     // ahead)
8     public Token peek()
9
10    // Consume and return next token
11    public Token read()
12
13    // -- Private helpers
14    -----
15
16    // Skip whitespace characters and // line comments
17    private void skipWhitespaceAndComments()
18
19    // Produce the next token from the source stream
20    private Token readInternal()
21
22    // Return true if c is a valid operator-symbol character
23    private boolean isOpSym(char c)
24 }

```

Listing 2: `Lexer.java` – public interface and internal helpers

Token categories recognised:

TokenType	Rule
IDENTIFIER	Letter followed by <code>[A-Za-z0-9_]*</code> ; not a keyword
INTEGER	One or more decimal digits
STRING	Single-quoted; raw escape sequences preserved
KEYWORD	Reserved words (see Table 1)
OPERATOR	Maximal sequence of operator symbols
PUNCTUATION	<code>() , ;</code>
EOF	End of input

let	in	fn	where	aug	or
not	gr	ge	ls	le	eq
ne	true	false	nil	dummy	within
and	rec				

Table 1: RPAL reserved keywords

4.3 Parser: `src/parser/Parser.java`

Recursive-descent parser that implements the full RPAL grammar. Nodes are accumulated on an internal stack; `build(label, n)` pops n children and pushes a new parent node.

```

1 public class Parser {
2
3     // Constructor: takes a fully constructed Lexer
4     public Parser(Lexer lex)
5
6     // Parse entire program; return root ASTNode
7     public ASTNode parse()
8
9     // -- Grammar rule methods (each leaves one node on stack)
10    ) -----
11
12    private void E()      // E -> 'let' D 'in' E | 'fn' Vb+
13                          //      '.' E | Ew
14    private void Ew()     // Ew -> T 'where' Dr | T
15    private void T()      // T -> Ta (',' Ta)+ => tau | Ta
16    private void Ta()     // Ta -> Ta 'aug' Tc | Tc          (
17                          //      left-assoc)
18    private void Tc()     // Tc -> B '->' Tc '|' Tc | B      (
19                          //      right-assoc)
20    private void B()      // B -> B 'or' Bt | Bt
21    private void Bt()     // Bt -> Bt '&' Bs | Bs
22    private void Bs()     // Bs -> 'not' Bp | Bp
23    private void Bp()     // Bp -> A (relop A)? | A
24    private void A()      // A -> ['+' | '-' ] At ('+' | '-' At
25                          //      ) *
26    private void At()     // At -> Af ('*' | '/' Af) *
27    private void Af()     // Af -> Ap '**' Af | Ap          (
28                          //      right-assoc)
29    private void Ap()     // Ap -> Ap '@' '<ID>' R | R
30    private void R()      // R -> R Rn | Rn                (

```

```

        gamma, left-assoc)
25  private void Rn()    // Rn -> <ID> | <INT> | <STR> | true
    / false / nil / dummy / '(' E ') '
26  private void D()    // D  -> Da 'within' D | Da
27  private void Da()   // Da -> Dr ('and' Dr)+ | Dr
28  private void Dr()   // Dr -> 'rec' Db | Db
29  private void Db()   // Db -> Vl '=' E | <ID> Vb+ '=' E |
    '(' D ') '
30  private void Vb()   // Vb -> <ID> | '(' Vl ') ' | '(' ') '
31
32  // -- Stack helpers
    -----
33
34  // Pop n nodes, push new parent with given label
35  private void build(String label, int n)
36
37  // Lookahead helpers
38  private boolean isKw(String s)
39  private boolean isOp(String s)
40  private boolean isPunc(String s)
41  private void    eatKw(String s)
42  private void    eatOp(String s)
43  private void    eatPunc(String s)
44  private boolean isRnStart()
45  }

```

Listing 3: Parser.java – function prototypes

Key design notes:

- Left-recursive rules (B, Bt, A, At, Ap, R) are implemented as `while` loops to avoid infinite recursion.
- `fn` lambda bodies require a literal `.` (period) as the body separator.
- The infix `@` operator builds a three-child `@(E1, N, E2)` node.

4.4 AST Node: `src/parser/ASTNode.java`

```

1  public class ASTNode {
2      public String      label;    // node tag or leaf
        value
3      public List<ASTNode> children; // ordered child list
4

```

```
5     public ASTNode(String label)                                //
        leaf
6     public ASTNode(String label, List<ASTNode> children)        //
        interior
7     public static ASTNode leaf(String label)                    //
        factory
8 }
```

Listing 4: ASTNode.java

Leaf label formats: <ID:x>, <INT:5>, <STR:'hi'>; interior labels match grammar rule names (gamma, lambda, let, ->, tau, ...).

4.5 Standardizer: src/standardizer/Standardizer.java

Post-order tree transformation: children are standardized before the parent rule fires. Eight rewrite rules are applied:

```
1 public class Standardizer {
2
3     // Entry point: recursively standardize n and return new
        root
4     public static ASTNode standardize(ASTNode n)
5
6     // Convenience factory: build a node with vararg children
7     private static ASTNode node(String label, ASTNode... kids
        )
8 }
```

Listing 5: Standardizer.java – function prototypes

Standardization rules (applied inside `standardize`):

Input pattern	Output
<code>let(=(X,E), P)</code>	<code>gamma(lambda(X,P), E)</code>
<code>where(P, =(X,E))</code>	<code>gamma(lambda(X,P), E)</code>
<code>fcn_form(F, V1..Vn, E)</code>	<code>=(F, lambda(V1, lambda(..., lambda(Vn, E)...))</code>
<code>lambda(V1..Vn, E) ($n > 1$)</code>	Nested single-arg lambdas
<code>within(=(X1,E1), =(X2,E2))</code>	<code>=(X2, gamma(lambda(X1,E2), E1))</code>
<code>and(=(Xi,Ei) ...)</code>	<code>=(, (X1..Xn), tau(E1..En))</code>
<code>rec(=(X,E))</code>	<code>=(X, gamma(<Y*>, lambda(X,E)))</code>
<code>@(E1, N, E2)</code>	<code>gamma(gamma(N,E1), E2)</code>

4.6 CSE Machine: `src/cse/CSEMachine.java`

Implements the Mark-1 CSE machine. Execution is split into two phases: *flattening* (ST $\rightarrow$ δ -lists) and *evaluation* (step loop).

```

1 public class CSEMachine {
2
3     // Constructor: flattens the standardized tree into delta
4     // lists
5     public CSEMachine(ASTNode st)
6
7     // Enable/disable step-by-step trace output
8     public void setTrace(boolean t)
9
10    // Print all control structures (delta lists) to stdout
11    public void printDeltas()
12
13    // Run the CSE machine; program output written to stdout
14    public void run()
15 }
```

Listing 6: CSEMachine.java – public interface

```

1     // -- Flatten phase
2     -----
3
4     // Allocate a new empty delta list; return its index
5     private int newDelta()
```

```

5
6  // Flatten the ST root into delta[0]
7  private void flatten(ASTNode root)
8
9  // Recursively flatten node n, appending symbols into
   delta[into]
10 private void flattenInto(ASTNode n, int into)
11
12 // Collect bound variable names from a lambda parameter
   node
13 private List<String> collectVars(ASTNode varNode)
14
15 // Strip "<ID:" prefix and trailing ">" from a leaf label
16 private String stripId(String lbl)
17
18 // -- Evaluation phase
   -----
19
20 // Process one control symbol: dispatch to helpers below
21 private void apply(Symbol t)
22
23 // Handle GAMMA: pop rator and rand, dispatch to
   applyGamma
24 private void applyGamma(Symbol rator, Symbol rand)
25
26 // Handle builtin function application (Print, Conc,
   Order, ...)
27 private void applyBuiltin(Symbol b, Symbol rand)
28
29 // Handle arithmetic / relational / boolean operator
   application
30 private void applyOp(String op)
31
32 // Structural equality check for 'eq' / 'ne'
33 private boolean eqSym(Symbol a, Symbol b)
34
35 // -- Output helpers
   -----
36
37 // Print a value to stdout (no newline), matching rpal.
   exe format

```

```

38     private void printValue(Symbol v)
39
40     // Render a symbol value as a string; unescape=true for
       output
41     public String valueStr(Symbol v, boolean unescape)
42
43     // Apply escape sequences (\n \t \\ \') to a raw string
44     private String unescape(String s)
45
46     // -- Trace
       -----
47
48     // Print C / S / env state for one step
49     private void traceStep(int step)
50
51     // Compact string representation of a symbol (for trace/
       deltas)
52     private String symStr(Symbol s)

```

Listing 7: CSEMachine.java – internal methods

Symbol model

```

1  public enum K {
2      NAME, INT, STR, BOOL, NIL, DUMMY, YSTAR,
3      LAMBDA, ETA, TUPLE, GAMMA, TAU, AUG,
4      BETA, DELTA_THEN, DELTA_ELSE,
5      OP, ENV_MARKER, BUILTIN
6  }
7
8  // Factory methods (static, on CSEMachine)
9  static Symbol mkInt(long v)
10 static Symbol mkStr(String v)
11 static Symbol mkBool(boolean v)
12 static Symbol mkName(String name)
13 static Symbol mkOp(String name)
14 static Symbol mkBuiltin(String name)
15 static Symbol mkNil()
16 static Symbol mkDummy()
17 static Symbol mkYstar()
18 static Symbol mkTuple(List<Symbol> items)

```

Listing 8: Symbol kinds and factory methods

Environment

```
1 public static class Environment {  
2     public final int id;           // unique env id (e0, e1,  
3     ...)  
4     public final Environment parent;  
5     public final Map<String, Symbol> bindings;  
6  
7     public Environment(int id, Environment parent)  
8  
9     // Walk parent chain; return binding or null if unbound  
10    public Symbol lookup(String name)  
11 }
```

Listing 9: Environment class

Flatten convention

For a **gamma** node, the *rand* subtree is flattened first, then the *rator* subtree, then the **GAMMA** symbol. This ensures the rator sits on top of the stack when the **GAMMA** symbol is executed (*rator-on-top* convention).

Key evaluation rules

Control top	Action
Value literal	Push onto S
NAME	Lookup in current env; push result onto S
LAMBDA	Capture <code>curEnv</code> ; push closure onto S
GAMMA	Pop rator, pop rand; call <code>applyGamma</code>
TAU	Pop n items; push tuple (reversed)
AUG	Pop val, pop tuple; append val to tuple
BETA	Pop bool; prepend chosen δ_{then} or δ_{else} to C
ENV_MARKER	Pop result; pop env marker from S ; restore parent env
Y^* applied to λ	Wrap as Eta closure
Eta applied to arg	Expand fixpoint: push two GAMMAs to C
Tuple applied to int i	Index tuple (1-based)

4.7 Tree Printer: `src/util/TreePrinter.java`

```

1 public class TreePrinter {
2
3     // Print the subtree rooted at n using dotted pre-order
4     // format.
5     // depth dots are prepended to each node label.
6     // Example: ".gamma" at depth 1, "..lambda" at depth 2.
7     public static void print(ASTNode n, int depth)
8 }

```

Listing 10: TreePrinter.java

Pre-order traversal: the node's own label is printed before any child is visited. This matches the `rpal.exe -ast` output format.

Built-in Functions

Name	Arity	Behaviour
Print	1	Print value; return <code>dummy</code>
Conc	2	Curried string concatenation
Stem	1	First character of string
Stern	1	String minus first character
Order	1	Tuple length (<code>nil</code> $\rightarrow$ 0)
Null	1	<code>true</code> if <code>nil</code> or empty tuple
ItoS	1	Integer $\rightarrow$ string
Isinteger	1	Type predicate
Istruthvalue	1	Type predicate
Isstring	1	Type predicate
Istuple	1	Type predicate
Isfunction	1	Type predicate
Isdummy	1	Type predicate

`Conc` is implemented as a curried builtin: the first application stores its argument in a `partial` field on the `Symbol`; the second application performs concatenation.

Build and Run

6.1 Building

```
make
```

Listing 11: Build with GNU Make

```
mkdir out -Force | Out-Null
javac -d out -sourcepath src rpal20.java '
  (Get-ChildItem -Recurse src -Filter *.java | % { $_.
    FullName })
```

Listing 12: Build with raw `javac` (Windows PowerShell)

6.2 Running

```
java -cp out rpal20 [switches] <file>
```

Listing 13: Command-line usage

Switch	Effect
(none)	Run program; print result. Matches <code>rpal.exe</code>
<code>-l</code>	Echo source before running
<code>-ast</code>	Print AST in dotted pre-order; exit
<code>-sast / -st</code>	Print Standardized Tree; exit
<code>-cse</code>	Print δ -lists + step trace; run

6.3 Make Targets

```
make run    FILE=rpal_test_programs/rpal_01
make ast    FILE=rpal_test_programs/rpal_03
make sast   FILE=rpal_test_programs/rpal_06
make cse     FILE=rpal_test_programs/rpal_01
make output FILE=rpal_test_programs/rpal_01 OUTPUT=result.txt
make clean
```

Listing 14: Makefile targets

The output target prints all four stages (result, AST, ST, CSE trace) to the terminal. If `OUTPUT=file` is supplied, output is simultaneously saved to that file via `tee`.

Sample Program and Output

```
// Factorial, tuple sum, string concat.

let rec fact n = n eq 0 -> 1 | n * fact (n-1)
in
let Sum T =
  let rec S i =
    i gr (Order T) -> 0 | T i + S (i+1)
  in S 1
in
let greet name = Conc 'Hello, ' (Conc name '!')
in
Print ( fact 5, Sum (10,20,30,40), greet 'RPAL' )
```

Listing 15: `rpal_test_programs/sample.rpal`

```
(120, 100, Hello, RPAL!)
```

Listing 16: Expected output

Testing

The interpreter output is compared against the reference `rpal.exe`:

```
java -cp out rpal20 rpal_test_programs/rpal_01 > mine.out
rpal.exe      rpal_test_programs/rpal_01 > ref.out
diff mine.out ref.out
```

Listing 17: Diff-based correctness check

A zero-line diff indicates a perfect match for that test case.